

# SECOND SPECIES AND INVERTIBLE COUNTERPOINT BY INTEGER PROGRAMMING

Katherine Guerrierio<sup>†</sup>

Johns Hopkins University  
Department of Computer Science

Alex Ma<sup>†</sup>

Johns Hopkins University  
Department of Computer Science

## ABSTRACT

Species counterpoint is a widely taught problem format in music theory. Typically, it deals with two voices: given a challenge melody (cantus firmus), one must compose a response melody (counterpoint) that adheres to certain rules. Numerous approaches to automatic species counterpoint composition have been proposed, including guided local search [1] [2], rule-based expert systems [3], Pareto optimality [4], and weighted refinement types [5]. In [6], Tanaka originated the use of *integer programming* for first species counterpoint. However, in that study, the question of whether the higher species admit efficient integer programming formulations was left open. At the same time, the literature lacks inquiry into the automatic composition of *invertible counterpoints*, cantus-counterpoint pairs that still work when swapped (roughly speaking). In this paper, we address these gaps by replicating Tanaka’s first species formulation, then extending it to additionally admit second species. Furthermore, we equip first species with options for invertibility at the octave. Finally, we verify and analyze our approach experimentally using short cantus firmi from Fux’s *Gradus ad Parnassum*, consider their musical qualities, and discuss factors affecting wall clock solve time and their implications for the complexity of second species compared to first.

## 1. INTRODUCTION AND RELATED WORK

Since the first computer-generated piece of music, Lejaren Hiller’s 1957 *Illiad Suite* ([7]), the computer-aided analysis and creation of music have flourished, turning into the modern disciplines of computational musicology and computer music. Objects of study have included counterpoint composition in the Palestrina style via Markov chains in [8]; chorale composition in the Bach style via deep long short-term memory in [9]; and species counterpoint composition

via local search in [1] and [2], and integer programming in [6].

The last problem format named above, species counterpoint, is a simplified, pedagogical version of real-world counterpoint — the study of writing for multiple voices. Species counterpoint was codified by composer Johann Joseph Fux in his 1725 *Gradus ad Parnassum*. The impact of species counterpoint on Common Practice composition in the West is hard to overstate — composers such as Mozart, Haydn, and Beethoven were known to have studied and taught from the *Gradus* extensively, and Mozart’s mentor Giovanni Battista Martini even declared in 1770 that “We have no system other than that of Fux” [10]. Today, versions of species counterpoint are typically taught in first-year undergraduate curricula at conservatories and universities around the world.

In its modern formulation, species counterpoint typically deals with only two voices — the student is given a challenge melody called a *cantus firmus*, and must respond with a valid response melody called a *counterpoint*. The two lines are meant to sound harmonious when played together while still retaining their sense of independence from one another. To train students to balance these opposing forces, species counterpoint proceeds in five levels of increasing complexity, or “species”:

1. First species deals with note-to-note relations; the counterpoint and cantus firmus are both whole notes, in a 1:1 ratio.
2. Second species introduces strong and weak beats; the counterpoint plays half notes while the cantus firmus plays whole notes, in a 2:1 ratio.
3. Third species further subdivides the measure; the counterpoint plays quarter notes while the cantus firmus plays whole notes, in a 4:1 ratio.
4. Fourth species deals with syncopations; the counterpoint and cantus firmus are both whole notes, but the counterpoint plays on weak beats while the cantus firmus plays on strong beats.
5. Fifth species combines all of the above.

<sup>†</sup>Equal contribution.

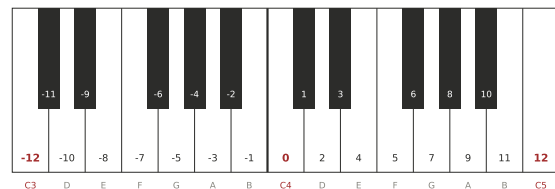


As mentioned previously, many approaches have been proposed for specifically species counterpoint. One such approach is integer programming, proposed by Tsubasa Tanaka for first species counterpoint. However, Tanaka notes that “Possible drawbacks of the proposed method are relatively large computational time and the complexity of programming many rules... it is beyond the scope of this paper to verify to which extent the integer-programming-based method is realistic to formulate more complex counterpoints” [6]. In other words, Tanaka raised the question of whether integer programming is realistic for the higher species. Are they too complex for integer programming to solve? In this paper, we show that while a natural integer programming formulation for second species does exist, many cases already require significantly more computational resources than first species, suggesting that the higher species are not amenable to integer programming solutions. In other words, existing expert-based systems are likely to outperform integer programming-based solutions for the higher species.

In addition to the above-mentioned gap, it was noted during preliminary research that no literature exists for automatic composition of invertible counterpoints. An invertible counterpoint is, roughly speaking, a pair of melodies that still works as a cantus-counterpoint pair when you swap them. Therefore, our second contribution in this paper is to introduce a formulation of invertibility to first species. This turns out to not add runtime to the integer program. However, it does lead to more cantus firmi being found infeasible, as the cantus firmus must also follow all the melodic rules of a counterpoint.

In this paper, we follow the example of the original paper and use PuLP, a Python package for linear and mixed-integer programming with interfaces into solvers such as Gurobi, SCIP, and COPT. Following the example of the original paper, we employ PuLP’s default open-source COIN-OR CBC solver. Musically speaking, we follow the conventions of most work in this area and consider only the case where we write counterpoints above the cantus firmus in the key of C Major (that is, the Ionian mode).

Rulesets for species counterpoint vary, from Palestrina-based Renaissance counterpoint (modal), to Bach-based Baroque counterpoint (tonal), to a more modern “period-agnostic” counterpoint. Therefore, in terms of ruleset, we primarily base our rules off of the original Fux. (Tanaka had used a French harmony textbook, apparently inaccessible in English.) Recognizing the element of subjectivity in music, we have also taken some liberties with regard to adjusting the rules to ensure aesthetically pleasing results. Our decisions for the ruleset should therefore not be taken as authoritative; instead, as has been done in other work, it should simply be construed as a simple, representative model upon which we may without loss of generality test our hypotheses.



**Figure 1.** Following Tanaka’s example, we encode notes using integers, with 0 representing middle C. We highly recommend referring back to this keyboard diagram to ground the reader’s understanding of our approach in actual pitches and intervals.



**Figure 2.** An example first species counterpoint as solved by our integer program — a successful replication of Tanaka’s results.

## 2. FIRST SPECIES COUNTERPOINT

First species deals with the note-to-note combination of voices. An example of a solved first species counterpoint can be found in Figure 2. For first species, we primarily use Tanaka’s formulation from [6]. However, we also supply a few corrections and adjustments to account for differing standardizations of species counterpoint. We reprise the essentials of Tanaka’s development below with our adjustments.

### 2.1 Constants and Decision Variables

Several constants and decision variables are used in all three of the problem formats studied. We summarize them below before listing the constraints.

#### 2.1.1 Cantus Firmus

The challenge melody is encoded as a list of integers  $CF_0, CF_1, \dots, CF_{T-1}$  representing the semitone offset of each note from middle C ( $C4 = 0$ ), where  $T$  is the number of bars. For example, the running example in our figures is  $CF = [0, 2, 5, 4, 7, 5, 4, 2, 0]$ , encoding scale degrees  $\hat{1} \hat{2} \hat{4} \hat{3} \hat{5} \hat{4} \hat{3} \hat{2} \hat{1}$  in C major (i.e., C, D, F, E, G, F, E, D, C). Negative integers represent notes below C4. Note that this paper uses 0-indexing.

#### 2.1.2 Index Sets

We use  $T_k = \{0, 1, \dots, T - 1 - k\}$  throughout, so that  $T_0$  is all bars,  $T_1$  is bars with at least one successor,  $T_2$  is bars with two successors, and so on. This helps with rules that govern  $k$ -grams.

### 2.1.3 Interval and Pitch Domains

Let  $H \subseteq \mathbb{Z}$  be the set of allowable harmonic intervals (in semitones above the cantus firmus), and let  $M = M^+ \cup M^-$  be the set of allowable melodic intervals in the counterpoint, with

$$M^+ = \overbrace{\{1, 2, 3, 4, 5, 7, 8, 12\}}^{\text{ascending melodic motion}},$$

$$M^- = \overbrace{\{-1, -2, -3, -4, -5, -7, -8, -12\}}^{\text{descending melodic motion}}.$$

The counterpoint pitch domain  $P = \{0, 2, 4, 5, 7, 9, 11, 12, 14, 16, 17, 19, 21\}$  corresponds to diatonic pitches in C major from C4 to A5.

For first species, we encode the set of allowable harmonic intervals as

$$H = \{0, 3, 4, 7, 8, 9, 12, 15, 16, 17, 19, 20, 21, 24\}, \quad (1)$$

which encodes all consonances (unisons, thirds, fifths, sixths, octaves, and their compounds up to two octaves) above the cantus firmus. The perfect fourth (5) is excluded because it is considered a dissonance in the common practice style unless it is hidden in the inner voices, which is impossible in a two-voice composition.

### 2.1.4 Decision Variables

The main decision variables are three families of binary indicator variables:

- $h_{t,i} \in \{0, 1\}$  for  $t \in T_0, i \in H$ : equals 1 iff the harmonic interval at bar  $t$  is  $i$  semitones.
- $m_{t,i} \in \{0, 1\}$  for  $t \in T_1, i \in M$ : equals 1 iff the melodic interval from bar  $t$  to bar  $t+1$  is  $i$  semitones.
- $p_{t,u} \in \{0, 1\}$  for  $t \in T_0, u \in P$ : equals 1 iff the counterpoint pitch at bar  $t$  is  $u$  semitones above C4.

We define integer-valued variables  $hInterval_t = \sum_{i \in H} i \cdot h_{t,i}$  and  $mInterval_t = \sum_{i \in M} i \cdot m_{t,i}$  to read out what the harmonic interval and melodic interval at time step  $t$  actually are. Auxiliary binary variables  $up_t, down_t, conjunct_t, contrary_t, turn_t$  are used to track melodic and inter-voice motion, and are defined to toggle on and off according to  $h_{t,i}, m_{t,i}$ , and  $p_{t,u}$ . Finally,  $consecutiveLeap_t \in \{0, 1, 2\}$  counts leaps in a two-bar window;  $largeLeap_t$  flags counterpoint leaps larger than a third; and  $climax_t$  together with the integer  $maxP$  identifies the unique highest counterpoint pitch via a standard big- $M$  encoding, as done in class.

### 2.1.5 Basic Encoding Constraints

At each time step, exactly one  $h_{t,i}, p_{t,u}$ , and  $m_{t,i}$  should be chosen at a time:

$$\sum_{i \in H} h_{t,i} = 1, \quad \sum_{u \in P} p_{t,u} = 1, \quad \sum_{i \in M} m_{t,i} = 1,$$

To these, we add constraints to define harmonic and melodic intervals:

$$\sum_{u \in P} u \cdot p_{t,u} = CF_t + hInterval_t,$$

$$mInterval_t = (CF_{t+1} + hInterval_{t+1}) - (CF_t + hInterval_t).$$

The above two equations ensure that harmonic intervals and melodic intervals have their natural musical meaning.

## 2.2 First Species Counterpoint Constraints

### 2.2.1 Consonant Intervals Only

$$\sum_{i \in H} h_{t,i} = 1 \quad \forall t \in T_0 \quad (2)$$

At every bar, the counterpoint must form a consonant interval with the cantus firmus. Together with the choice of  $H$  above, this excludes seconds, fourths, sevenths, tritones, and their compound interval versions.

### 2.2.2 Unisons Only at the Beginning and End

$$h_{t,0} = 0 \quad \forall t \in \{1, 2, \dots, T-2\} \quad (3)$$

The unison is treated as too perfect (i.e. finished-sounding) to occur in the middle of a piece; it is allowed only at the opening and final bars.

### 2.2.3 No Parallel Perfect Intervals

$$h_{t,i_1} + h_{t+1,i_2} \leq 1 \quad \forall (i_1, i_2) \in \mathcal{P}_8 \cup \mathcal{P}_5 \quad (4)$$

where  $\mathcal{P}_8 = \{0, 12, 24\}^2$  is the set of pairs of octave-class intervals and  $\mathcal{P}_5 = \{7, 19\}^2$  is the set of pairs of fifth-class intervals. This prevents the two voices from forming any pair of consecutive perfect consonances of the same class (including, e.g., a unison followed by an octave or a fifth followed by a twelfth). In traditional counterpoint, parallel perfect intervals cause the two voices to lose their sense of independence.

### 2.2.4 No Hidden (Direct) Perfect Intervals

$$h_{t+1,i} \leq 1 - \text{hiddenTrigger}_t \quad \forall i \in \{0, 7, 12, 19, 24\} \quad (5)$$

A hidden interval occurs when both voices move in the same direction and land on a perfect interval, with the upper voice arriving by leap (it is, however, permissible for the upper voice to arrive by a step). Here  $\text{hiddenTrigger}_t$  is a binary auxiliary equal to  $\text{similar}_t \wedge \neg \text{conjunct}_t$ , where  $\text{similar}_t \in \{0, 1\}$  is equal to 1 if and only if the counterpoint and cantus firmus are both moving up or both moving down melodically at time step  $t$ .

### 2.2.5 No Arpeggiation of Triads

$$m_{t,3} + m_{t,4} + m_{t+1,3} + m_{t+1,4} \leq 1 \quad (6)$$

This prevents the melody from outlining a triad through consecutive same-direction leaps. The full constraint family adds five more pattern-specific cases for triad recognition in accordance with Tanaka’s rules.

### 2.2.6 No Consecutive Leaps in Same Direction

$$\text{consecutiveLeap}_t \leq 1 + \text{turn}_t \quad (7)$$

Two consecutive leaps are only allowed if the melody changes direction between them. In other words, it is forbidden to leap twice in the same direction consecutively.

### 2.2.7 Limit Pitch Repetition

$$\sum_{s=t}^{t+4} p_{s,u} \leq 2 \quad \forall t \in T_4, u \in P \quad (8)$$

A single pitch cannot occur more than twice within any sequence of five consecutive bars. This ensures pitch variety and a sense of motion.

### 2.2.8 No Simultaneous Large Leaps

$$\text{largeLeap}_t \leq \text{contrary}_t \quad (9)$$

If the cantus firmus makes a large leap (greater than a third), the counterpoint may only do so if it moves in the opposite direction.

### 2.2.9 Large Leap Compensation

$$m_{t+1,\pm 1} + m_{t+1,\pm 2} \geq \text{largeLeap}_t \quad (10)$$

After a large leap, the melody should move stepwise in the opposite direction. While Fux requires this, our generated counterpoints sounded melodious without it. In any case, Tanaka’s formulation did not implement this rule — indeed, he noted that rule 2.2.6 against consecutive leaps in the same direction was sufficient, but we noted a simple counterexample. In scale degrees, the melody  $\hat{1}\hat{5}\hat{6}$  (i.e. C-G-A) would not be disallowed by rule 2.2.6 as it is a leap followed by a step in the same direction.

### 2.2.10 No Voice Crossing

$$h\text{Interval}_t \geq 0 \quad \forall t \in T_0 \quad (11)$$

The pitch of the counterpoint must remain at or above the pitch of the cantus firmus at all times. (Because linear programs restrict  $H$  to non-negative intervals, this constraint is already implicit in our formulation.)

### 2.2.11 Limit Parallel Thirds and Sixths

$$\sum_{s=t}^{t+3} (h_{s,3} + h_{s,4}) \leq 3, \quad \sum_{s=t}^{t+3} (h_{s,8} + h_{s,9}) \leq 3 \quad (12)$$

$$\sum_{s=t}^{t+3} (h_{s,15} + h_{s,16}) \leq 3, \quad \sum_{s=t}^{t+3} (h_{s,20} + h_{s,21}) \leq 3 \quad (13)$$

No more than three consecutive thirds, sixths, tenths, or thirteenths are allowed. While these intervals are euphonious, four or more in a row gives the impression of two voices moving as one rather than as two independent voices.

### 2.2.12 Opening Interval Must Be Perfect Unison, Fifth, or Octave

$$h_{0,0} + h_{0,7} + h_{0,12} = 1 \quad (14)$$

The first interval must be a unison, fifth, or octave. This establishes a tonic anchor as “home” for the piece. Note that while our setting concerns the counterpoint above the cantus firmus, it is also possible to write the counterpoint below the cantus firmus. In that setting, the only allowable opening intervals would be a unison and octave, for a fifth would tonicize the subdominant  $\hat{4}$  (i.e. F) in the counterpoint rather than the intended tonic  $\hat{1}$  (i.e. C) in the cantus firmus. In the invertible variant, 4.2.0.3 encodes that rule.

### 2.2.13 Penultimate Note Approaches the Final by Step

$$m_{T-2,-2} + m_{T-2,-1} + m_{T-2,1} + m_{T-2,2} = 1 \quad (15)$$

The melodic motion into the final bar must be stepwise (a leading tone  $\hat{7}\hat{1}$  or supertonic  $\hat{2}\hat{1}$  resolution; i.e. B-C or D-C).

### 2.2.14 Final Interval Must Be Unison or Octave

$$\sum_{i \in \{0,12,24\}} h_{T-1,i} = 1 \quad (16)$$

The piece must end on either a unison, octave, or two octave harmonic interval. We implement this in the code through an interval-class encoding, which generalizes easily to compound intervals.

### 2.2.15 Global Melodic Quality

#### 2.2.15.1 Contrary Motion

$$\sum_{t \in T_1} \text{contrary}_t \geq \frac{T}{2} \quad (17)$$

#### 2.2.15.2 Stepwise Motion

$$\sum_{t \in T_1} \text{conjunct}_t \geq \frac{T}{2} \quad (18)$$



**Figure 3.** An example second species counterpoint as solved by our integer program.

### 2.2.15.3 Unique Climax

$$\sum_{t \in T_0} \text{climax}_t = 1 \quad (19)$$

The above constraints enforce global musical shape. Contrary motion gives sufficient independence between voices, stepwise movement gives smooth melodic motion, and a unique climax pitch (i.e. highest pitch; enforced via big- $M$  using *maxP*) gives direction to the melody.

## 2.3 Optimization Objective for First Species

Tanaka proposed a simple objective that minimizes melodic turns while rewarding contrary and stepwise motion:

$$\min \sum_{t \in T_2} (\text{turn}_t - \text{contrary}_t - \text{conjunct}_t). \quad (20)$$

This was used as a general stand-in for objective functions, allowing the investigation of integer programming itself without loss of generality to other objective functions. Therefore, we retain it. The sums are taken over  $T_2$  for indexing consistency among all three terms; the difference relative to summing over  $T_1$  is at most one term and is musically negligible.

## 3. SECOND SPECIES COUNTERPOINT

Second species counterpoint has a 2:1 note duration ratio, adding to first species the idea of a *strong* beat (or “downbeat”) and a *weak* beat (or “upbeat”) in every measure. Parallel fifths, octaves, and unisons are still forbidden. However, there is an additional consideration — parallel fifths, octaves, and unisons from one downbeat to the next are also forbidden. We also allow dissonance, but only on weak beats and only when handled as passing tones. An example solved second species counterpoint instance can be found in Figure 3.

We index counterpoint subbeats  $s = 0, 1, \dots, S - 1$ , where  $S = 2N - 1$  for an  $N$ -bar cantus firmus (so the final bar contains a single whole-note counterpoint, while every other bar contains two half notes). We write  $\text{Downbeats} = \{s \in S_0 : s \text{ even}\}$ ,  $\text{Upbeats} = \{s \in S_0 : s \text{ odd}\}$ , and define  $CF_s = CF_{\lfloor s/2 \rfloor}$  as the cantus firmus pitch active at subbeat  $s$ .

The harmonic interval set is split into consonant and dissonant intervals, namely:

$$H_{\text{CONS}} = \{0, 3, 4, 7, 8, 9, 12, 15, 16, 19, 20, 21, 24\},$$

$$H_{\text{DISS}} = \{1, 2, 6, 10, 11, 13, 14, 18, 22, 23\},$$

with  $H = H_{\text{CONS}} \cup H_{\text{DISS}}$ .

## 3.1 Second Species Counterpoint Constraints

### 3.1.1 Downbeats Must Be Consonant

$$\sum_{i \in H_{\text{CONS}}} h_{s,i} = 1 \quad \forall s \in \text{Downbeats} \quad (21)$$

Every strong beat (downbeat) must form a consonant interval with the cantus firmus. The downbeat is metrically stable and defines the harmonic structure. Thus, dissonances are disallowed here due to their harmonic instability.

### 3.1.2 Upbeats May Be Dissonant

$$\text{isDissonant}_s = \sum_{i \in H_{\text{DISS}}} h_{s,i} \quad \forall s \in \text{Upbeats} \quad (22)$$

Second species does allow dissonance, but only in weak beats where it is less disruptive to the implied harmony.

### 3.1.3 Downbeats Are Never Dissonant

$$\text{isDissonant}_s = 0 \quad \forall s \in \text{Downbeats} \quad (23)$$

Dissonance is explicitly forbidden on strong beats, which reinforces the consonant-downbeat rule and ensures all structural harmonic points remain stable. This rule may be redundant due to rule 3.1.1, but we hoped to avoid premature optimization, allowing the solver to presolve and optimize its efforts instead.

### 3.1.4 Dissonances Must Be Passing Tones: Stepwise Approach

$$\text{conjunct}_{s-1} \geq \text{isDissonant}_s \quad (24)$$

If a note is dissonant, it must be approached stepwise.

### 3.1.5 Dissonances Must Be Passing Tones: Stepwise Resolution

$$\text{conjunct}_s \geq \text{isDissonant}_s \quad (25)$$

A dissonant note must also leave by step. This ensures that the dissonance resolves in a natural way rather than continuing to build uncontrolled harmonic tension.

### 3.1.6 Passing Tones Must Continue in Same Direction

$$up_{s-1} - up_s \leq 1 - \text{isDissonant}_s, \quad up_s - up_{s-1} \leq 1 - \text{isDissonant}_s \quad (26)$$

The motion into and out of a dissonant note must continue in the same direction (true passing-tone motions are allowed and neighboring tones are disallowed).

### 3.1.7 Opening Interval Must Be Perfect

$$h_{0,0} + h_{0,7} + h_{0,12} = 1 \quad (27)$$

As in first species, the first interval must be a unison, fifth, or octave.

### 3.1.8 Penultimate Note Must Approach by Step

$$m_{S-2,-2} + m_{S-2,-1} + m_{S-2,1} + m_{S-2,2} = 1 \quad (28)$$

The final melodic motion must be stepwise.

### 3.1.9 No Interior Downbeat Unisons

$$h_{s,0} = 0 \quad \forall s \in \text{Downbeats}, s \notin \{0, S-1\} \quad (29)$$

Unisons on downbeats are only allowed at the beginning and end.

### 3.1.10 No Parallel Perfect Intervals

$$h_{2b,i_1} + h_{2(b+1),i_2} \leq 1 \quad (30)$$

$$h_{2b+1,i_1} + h_{2(b+1),i_2} \leq 1 \quad (31)$$

for all pairs  $(i_1, i_2) \in \{0, 12, 24\}^2 \cup \{7, 19\}^2$  (octave-class or fifth-class). Eq. (24) blocks parallel perfects between successive downbeats. Eq. (25) blocks parallel perfects across the barline from an upbeat into the following downbeat (which is the only upbeat-to-downbeat boundary at which the cantus firmus actually moves).

### 3.1.11 No Hidden Perfect Intervals into Downbeats

$$h_{2(b+1),i} \leq 1 - \text{hiddenTrigger}_b \quad \forall i \in \{0, 7, 12, 19, 24\} \quad (32)$$

where  $\text{hiddenTrigger}_b = \text{similarDownbeat}_b \wedge \neg \text{conjunct}_{2b+1}$ , and  $\text{similarDownbeat}_b$  matches the counterpoint's direction at  $s = 2b + 1$  to the cantus firmus's direction from bar  $b$  to  $b+1$ . In second species, this rule only applies to approaches to downbeats.

### 3.1.12 No Triadic Arpeggiation

$$m_{s,3} + m_{s,4} + m_{s+1,3} + m_{s+1,4} \leq 1 \quad (33)$$

The melody cannot outline a chord through consecutive leaps. (As with first species, we input a six-pattern family covering all triadic outlines; this is just one example.)

### 3.1.13 No Consecutive Leaps in Same Direction

$$\text{consecutiveLeap}_s \leq 1 + \text{turn}_s \quad (34)$$

Two leaps in the same direction are only allowed if the melody changes direction between them.

### 3.1.14 Limit Pitch Repetition

$$\sum_{k=s}^{s+4} p_{k,u} \leq 2 \quad (35)$$

A pitch may not occur more than twice in five consecutive subbeats.

### 3.1.15 Avoid Simultaneous Large Leaps

$$\text{largeLeap}_s \leq \text{contrary}_s \quad \forall s \text{ s.t. CF leaps at } s \quad (36)$$

If the cantus firmus leaps into a downbeat, the counterpoint may only leap simultaneously if the motion is contrary.

### 3.1.16 Limit Parallel Thirds and Sixths

$$\sum_{s \in W} (h_{s,3} + h_{s,4} + h_{s,15} + h_{s,16}) \leq 3 \quad (37)$$

$$\sum_{s \in W} (h_{s,8} + h_{s,9} + h_{s,20} + h_{s,21}) \leq 3 \quad (38)$$

for  $W$  a window of four consecutive downbeats. No more than three consecutive thirds (or sixths) are allowed across downbeats. (Compound and simple intervals are both considered.) This is the second-species version of rule 2.2.11.

### 3.1.17 Encourage Stepwise Motion

$$\sum_s \text{conjunct}_s \geq \text{MinConjunct}, \quad \text{MinConjunct} = \lfloor S/2 \rfloor \quad (39)$$

The melody should move stepwise on at least half of subbeat transitions.

### 3.1.18 Encourage Contrary Motion

$$\sum_{s \in S_{\text{move}}} \text{contrary}_s \geq \text{MinContrary} \quad (40)$$

where  $S_{\text{move}} = \{s \in S_1 : CF_{s+1} \neq CF_s\}$  is the set of subbeat transitions across which the cantus firmus actually moves, and  $\text{MinContrary} = \max(1, \lfloor N/3 \rfloor)$ . This avoids spuriously inflating the contrary-motion count via the many subbeats where the cantus firmus is held.

### 3.1.19 Unique Climax

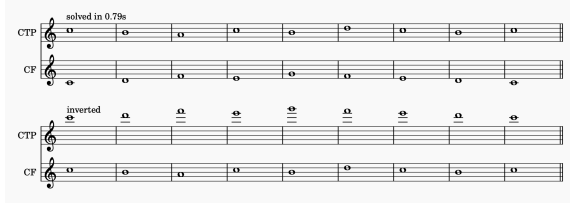
$$\sum_s \text{climax}_s = 1 \quad (41)$$

The melody must have exactly one highest point, enforced via the same big- $M$  encoding as in first species.

### 3.1.20 Closing Interval Must Be Octave or Unison

$$\sum_{i \in \{0, 12, 24\}} h_{S-1,i} = 1 \quad (42)$$

The final interval must be a unison or octave with the cantus firmus. (As in first species, fifths are not allowable ending harmonic intervals.)



**Figure 4.** An example invertible first species counterpoint as solved by our integer program.

### 3.2 Optimization Objective for Second Species

$$\min \sum_{s \in S_2} \text{turn}_s - \sum_{s \in S_1} \text{conjunct}_s - \sum_{s \in S_{\text{move}}} \text{contrary}_s \quad (43)$$

The objective again minimizes turns while rewarding stepwise and contrary motion, with contrary motion summed only over the subbeat transitions where the cantus firmus moves.

## 4. INVERTIBLE FIRST SPECIES COUNTERPOINT

One advanced form of counterpoint, not typically taught in introductory courses, is that of *invertible counterpoint*. Real-world examples abound: an interactive analysis of J.S. Bach’s Fugue BWV 885 can be found in [11]. In it, the subject and countersubject are played (a) against each other, then (a) inverted at the octave, then (b) inverted at the 12th, then (c) inverted at the 13th. Such an advanced contrapuntal texture that works at the octave, 12th, and 13th at the same time is beyond the scope of this article (and indeed beyond the scope of most music theory courses). Instead, we focused on enforcing invertibility at the octave. We tested these aurally by shifting the cantus firmus up two octaves to hear the inverted counterpoint. One may also see this as swapping the upper and lower voices, so that the cantus firmus is now on top and the counterpoint is now on the bottom. Thus, this is really a symmetry constraint, much like many others we have encountered in Declarative Methods. Unlike most symmetry constraints in this course, however, invertibility at the octave allowed for *faster* solve times.

We enforce invertibility at the octave through two complementary mechanisms: (i) a pre-validation pass on the cantus firmus that ensures it independently satisfies the counterpoint melodic rules, and (ii) a restricted harmonic interval set that is closed under octave inversion. We discuss each in turn.

### 4.1 Cantus Firmus Pre-Validation

In standard species counterpoint, the cantus firmus is treated as fixed input and need not itself satisfy the counterpoint melodic rules. For invertibility, however, the cantus firmus and counterpoint swap roles, so the cantus firmus must *also* satisfy every melodic rule we apply to a counterpoint voice. Because the cantus firmus is treated as fixed input rather than

a decision variable in our integer program, we cannot encode this requirement as additional constraints; instead, we pre-filter the cantus firmus list using a validator that checks for each of the melodic rules. A cantus firmus is rejected (and reported with a list of specific violations) if it fails any of the following:

- Every melodic interval lies in  $M$  (no augmented seconds, no repeated pitches, no leaps larger than an octave).
- The penultimate note approaches the final by step ( $|CF_{T-1} - CF_{T-2}| \in \{1, 2\}$ ).
- No two consecutive leaps in the same direction.
- No triadic arpeggio outlines (the same six patterns enforced on the counterpoint).
- At least  $\lfloor T/2 \rfloor$  stepwise melodic intervals.
- A unique climax pitch.
- No pitch repeated more than twice in any 5-bar window.

We found that several common of our experimental cantus firmi, particularly those that end with a 3–1 leap (i.e. E–C), fail the penultimate-stepwise check, and so are not invertible counterpoints.

### 4.2 Invertibility Constraints

Because the two voices may be swapped, every harmonic interval  $h$  must invert to a consonant harmonic interval. Inverting a voice up by 24 semitones sends  $h \mapsto 24 - h$ . Examining the consonance set:

- Unisons and octaves are stable:  $0 \leftrightarrow 24, 12 \leftrightarrow 12$ .
- Thirds become sixths and vice versa:  $3 \leftrightarrow 21, 4 \leftrightarrow 20, 8 \leftrightarrow 16, 9 \leftrightarrow 15$ .
- Fifths become fourths:  $7 \mapsto 17$ , but considering these modulo 12 gives  $7 \leftrightarrow 5$  (a perfect fourth, dissonant). Thus fifths are not allowed.

We therefore restrict the harmonic interval set to

$$H_{\text{inv}} = \{0, 3, 4, 8, 9, 12, 15, 16, 20, 21, 24\}, \quad (44)$$

which excludes fifths and is closed under the inversion map  $h \mapsto 24 - h$ .

The remaining first-species rules are kept, with three small adjustments:

#### 4.2.0.1 No interior unisons or octaves.

$$h_{t,0} = h_{t,12} = h_{t,24} = 0 \quad \forall t \in \{1, \dots, T-2\} \quad (45)$$

Since unisons and octaves are now permitted only at the endpoints due to rule 2.2.2 and the fact that octaves invert to unisons, we extend the no-interior-unisons rule to the entire octave class.

#### 4.2.0.2 Parallel and hidden octaves.

The parallel-perfects rule from Eq. (4) is restricted to octave-class pairs (since fifths are no longer in  $H_{\text{inv}}$ ). Similarly, the hidden-perfects rule from Eq. (5) is restricted to  $i \in \{0, 12, 24\}$ .

#### 4.2.0.3 Endpoints Must Be Unison-Class.

Both the opening and final intervals must be unison-class harmonic intervals:

$$\sum_{i \in \{0, 12, 24\}} h_{0,i} = 1, \quad \sum_{i \in \{0, 12, 24\}} h_{T-1,i} = 1. \quad (46)$$

Finally, the objective and all other constraints are unchanged from first species. Our experiments (Section 5) suggest that, despite the additional imposition of a symmetry constraint, invertibility actually decreases solver time on the cantus firmi that pass pre-validation; we discuss possible reasons in the following sections.

## 5. EXPERIMENTS

We ran 25 cantus firmi through first species, second species, and invertible first species, collating their outputs into MuseScore files. We created a translator (`translator.py`) to export the integer-encoded outputs into human-readable score notation, including double barlines and system breaks between examples and a text annotation showing the solver wall time per cantus firmus. To see and hear the musical results, download MuseScore (the Muse Hub add-on is also free, but is not needed), a free, open-source music notation application at <https://www.musescore.org>. Then, go to [https://drive.google.com/drive/folders/19dHpovI9uCfac7X\\_wmLyYhZ6JXiVR7Ig?usp=sharing](https://drive.google.com/drive/folders/19dHpovI9uCfac7X_wmLyYhZ6JXiVR7Ig?usp=sharing) to visit the experimental data folder and download the MuseScore files. That folder also includes our CSV files and charts, which provide insight into the steps the solver took to solve each problem.

Our 25 cantus firmi were all valid cantus firmi; that is, they began and ended on the tonic note, C. Most of them spanned about a perfect 5th or so. All cantus firmi were solved in less than 3.45s in first species. Not all cantus firmi were solvable in second species, though, with one timing out of our 10-minute time limit. For those that were feasible, the longest run took 565.74s (9m 25.74s) and the shortest run took 5.76s. Finally, as expected, invertible first counterpoint outright rejected the most cantus firmi, since that format requires cantus firmi to additionally adhere to the melodic rules for counterpoints. However, the addition of an invertibility constraint seemed to counterintuitively *reduce* the runtime, with the shortest run taking 0.3s to solve and the longest run taking 1.84s to complete. This was surprising because typically, symmetry constraints make problems harder, not easier! We speculate that a bit of survivor bias

is at play here: perhaps our pre-validation check for invertible first species rejects specifically those cantus firmi that are uncooperative to counterpoint writing.

### 5.1 Adjusting the Objective Function

The objective function ratio of (turns):(contrary):(conjunct) is currently 1 : -1 : -1, so the optimizer is indifferent between an additional contrary motion and an additional stepwise interval. One might expect that increasing the weight on `conjunct` produces smoother, more chant-like counterpoints, while increasing the weight on `contrary` leads to more independent motion at the expense of smoothness. We modified the coefficients on the objective function and ran it a few times, primarily focused on first species and invertible first species due to their fast runtime. While the counterpoints selected did change, there was no appreciable difference in the musical quality of the different counterpoints. In each case, all of the constraints were respected, so each counterpoint was still a valid solution. You can try this out for yourself by going to lines 472-476 in `first_species_ilp.py`, lines 399-404 in `invertible_first_species.py`, and lines 422-425 in `second_species_ilp.py`. Edit the coefficients, then run `python3 first_species_ilp.py` (or with corresponding file) in the correct directory. If you have `music21` (or have run `pip install -r requirements.txt` and `Musescore` installed, each solution will come out rendered in an open Musescore window after all experimental cantus firmi are solved (or marked infeasible). You can then press play in Musescore to hear the new results.

## 6. EXPERIMENTAL RESULTS AND ANALYSIS

We evaluate the integer linear programming (ILP) formulation of first species, invertible first species, and second species counterpoint across a range of cantus firmi (CF). Each experiment records solver performance, structural properties of the optimization problem, and musical characteristics of the resulting counterpoint.

In first species, all cantus firmi were solved to optimality in less than 2s using the CBC solver. This means we have successfully replicated Tanaka’s integer programming formulation with some corrections and adjustments.

### 6.1 Representative Results

Table 1 summarizes representative runs in first species. We report cantus firmus (CF), length, runtime characteristics (via iterations/nodes proxies), and key musical metrics.

### 6.2 Initial Solver Takeaways

Among the three species, second species was by far the most demanding. CBC’s presolve, in particular, dramatically reduced first-species and invertible-first-species prob-



| CF                                         | Len | Rows | Cols | Conj. | Contrary | Turns |
|--------------------------------------------|-----|------|------|-------|----------|-------|
| [0, -5, -7, -8, -7, -10, -5, -8, -10, -12] | 10  | 753  | 593  | 7/9   | 8/9      | 2     |
| [0, 2, 4, 2, 5, 7, 5, 4, 2, 0]             | 10  | 751  | 593  | 7/9   | 6/9      | 3     |
| [0, 4, 5, 7, 4, 5, 2, 0]                   | 8   | 575  | 469  | 5/7   | 6/7      | 3     |
| [0, 2, 0, 4, 5, 4, 2, 0]                   | 8   | 575  | 469  | 6/7   | 5/7      | 3     |
| [0, 7, 9, 7, 5, 4, 2, 0]                   | 8   | 576  | 469  | 5/7   | 5/7      | 3     |

**Table 1.** Representative first species results. Conjunct (stepwise) melodic intervals and contrary motion between voices are generally considered aesthetically preferable. Each is reported as a proportion of the total melodic intervals.

lem sizes (often  $\geq 70\%$ ), but had less of an effect on second species, where the doubled subbeat resolution and the cross-barline dissonance handling generated many tightly-coupled cuts. We attribute this to the additional auxiliary variables (*isDissonant*, *similarDownbeat*, the upbeat-to-downbeat parallel checks) which substantially raise the constraint count without yielding correspondingly tighter LP relaxations.

As is shown in Figures 5, 6, and 7, there is a reasonable fit to a power law (which may be more clearly borne out or rejected with more data), with scaling factors  $N^{2.60}$ ,  $N^{3.48}$ , and  $N^{5.84}$  for invertible first species, first species, and second species, respectively. Interpreting the power law, this means that in first species, a length- $2N$  cantus firmus will take about  $2^{3.48} \approx 11.16$  times the amount of time as a length- $N$  cantus firmus. In other words, doubling the length of the problem leads to a  $11.16\times$  increase in wall clock solve time. For second species, that scaling factor jumps to  $2^{5.84} = 57.28$ , which is much worse! This evidence seems to suggest that second species is fundamentally much more computationally difficult for integer programming than first species. We suggest that the higher species, especially fifth species, where one may choose to write in any of the first four species for a given bar, will prove especially problematic for integer programming. We note in 6.4.4.1 that while the number of notes in each bar accounts for most of the increased complexity of second species compared to first species, it does not account for all of it.

### 6.3 Computational Properties

Problem size scales with the length of the cantus firmus. For first species:

- 8-note CF:  $\sim 575$  rows,  $\sim 469$  columns
- 10-note CF:  $\sim 750$  rows,  $\sim 593$  columns

Invertible counterpoint reduces presolved problem size significantly (e.g. 159 rows from 695), suggesting stronger structural redundancy elimination during presolve. In contrast, second species approximately doubles the problem size (e.g. 1546 rows, 1399 columns), a seeming result of the approximately doubled rhythmic resolution. Note that third species would double that resolution once more.

The LP relaxation values (e.g.  $-14.0$  to  $-24.8$ ) indicate a relatively tight relaxation, though cutting planes are still required to close the optimality gap.

### 6.4 Musical Analysis

#### 6.4.1 Conjunct Motion

Across all experiments, conjunct motion consistently accounts for approximately 50%–75% of melodic intervals. For example, CF [0, -5, -7, -8, -7, -10, -5, -8, -10, -12]: 7/9 conjunct motions.

This aligns with traditional counterpoint guidelines favoring stepwise motion while allowing expressive leaps.

#### 6.4.2 Contrary Motion

Contrary motion is consistently high with up to 8/9 in longer CFs and typically  $\geq 70\%$  across all runs.

This confirms that the optimization strongly favors independence between voices, a central principle of species counterpoint and the common practice style more generally.

#### 6.4.3 Melodic Variety (Turns)

The number of melodic turns (direction changes) remains moderate with around 2–3 for first species. There is higher variability in second species due to increased rhythmic resolution. This suggests that the model is struggling to balance continuing motion in one direction with excessive fragmentation of the melodic line (via turns).

#### 6.4.4 Climax Behavior

Each generated counterpoint exhibits a unique climax pitch, consistent with imposed constraints. Climax pitches vary across CFs (e.g. 12, 14, 16, 19), indicating that the model adapts melodic range dynamically rather than fixing a pre-determined apex.

##### 6.4.4.1 Second Species vs. First Species

Second species demonstrates considerably increased problem complexity when compared to first species, as there were about 2 times as many variables and constraints. It also introduced dissonant upbeats (e.g. 2/9, 3/9 cases). Noting that the power law for first species varies proportional to  $N^{3.48}$ , and that second species has just about double the

number of notes as first species, one might expect second species of length  $N$  to behave like a first species problem of length  $2N$ . In other words, one might expect the wall clock time taken to vary as follows:  $\text{time}_2(N) \approx \text{time}_1(2N) = (2N)^{3.48} = 2^{3.48} \cdot N^{3.48}$ . This has a constant factor of about 11, but retains the same exponent. Instead, the observed scaling is  $N^{5.84}$ , an exponent roughly 2.36 higher than the naive prediction. This suggests that either (a) the limited length of cantus firmi encountered in our experiments (and in real usage) means that the constant factor actually represents quite a significant proportion of the increase in complexity, or (b) the doubling of note count alone cannot account for the increase in wall clock solve time. Rather, the additional dissonance handling and strong-beat-weak-beat considerations actually do contribute additional factors of  $N$  to the complexity of the integer programming formulation. In other words, we suggest that the disparity in apparent complexity cannot be accounted for by the increase in notes alone; instead, the addition of strong and weak beats and dissonance-handling really do add something “extra” over top of that which is due to an increase in length.

### 6.5 Effect of Cantus Firmus Structure

The structure of the cantus firmus strongly influences both solution quality and computational effort. Stepwise CFs (e.g.  $[0, 2, 4, 2, 5, 7, 5, 4, 2, 0]$ ) produce balanced motion and stable optimization behavior while leap-heavy CFs increase variability in contrary motion and solver effort. This suggests that some cantus firmi are more “cooperative” or “easier” than others to solve.

### 6.6 Summary

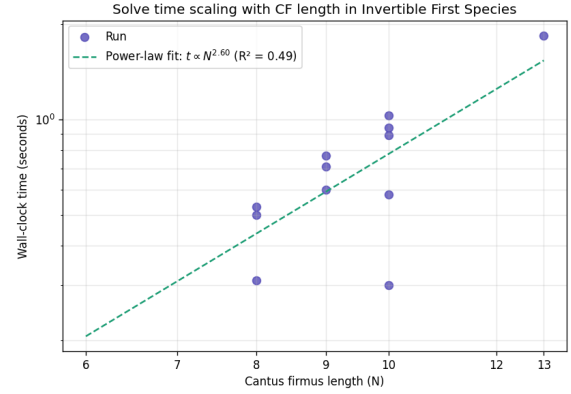
The ILP framework successfully generates stylistically valid counterpoint across multiple species. Key findings include:

- High rates of contrary motion ( $\geq 70\%$ )
- Balanced conjunct motion (50%–75%)
- Predictable scaling of computational complexity with CF length and species type

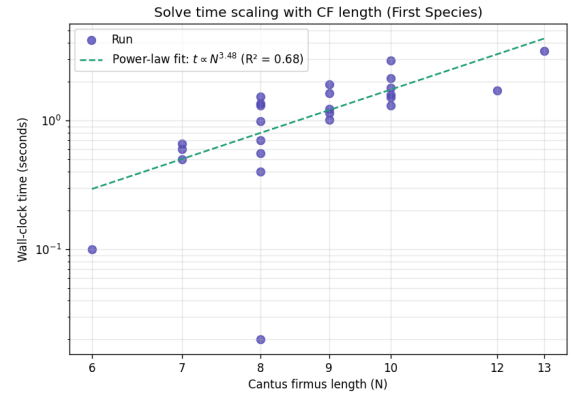
These results demonstrate that integer programming provides a robust and flexible framework for modeling traditional counterpoint rules while enabling quantitative evaluation of musical structure.

## 7. CONCLUSION

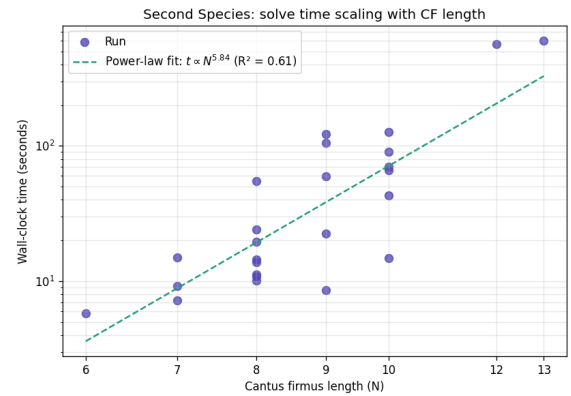
We have replicated, slightly corrected, and extended Tanaka’s integer-programming formulation of first species counterpoint, contributing (i) a natural extension to second species and (ii) a novel formulation of invertibility at the octave for first species. Empirically, we find that first species and invertible first species are well within reach of an open-source CBC solver (sub-second to a few seconds per cantus



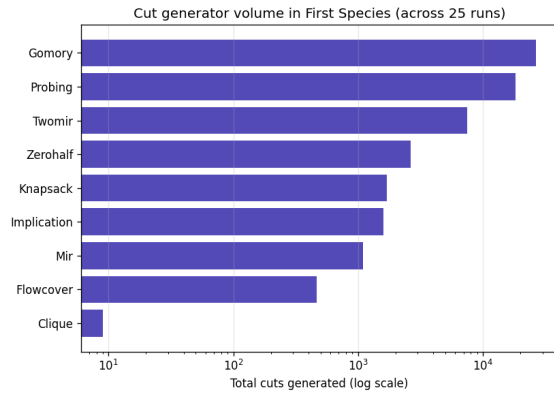
**Figure 5.** In invertible first species, the time taken to solve each CF scales proportional to  $N^{2.60}$  following a power law.



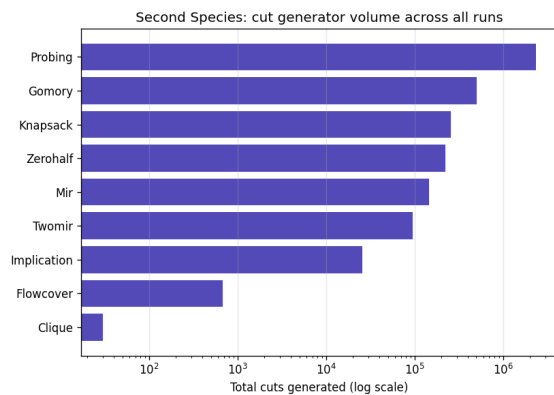
**Figure 6.** In first species, the time taken to solve each CF scales proportional to  $N^{3.48}$  following a power law.



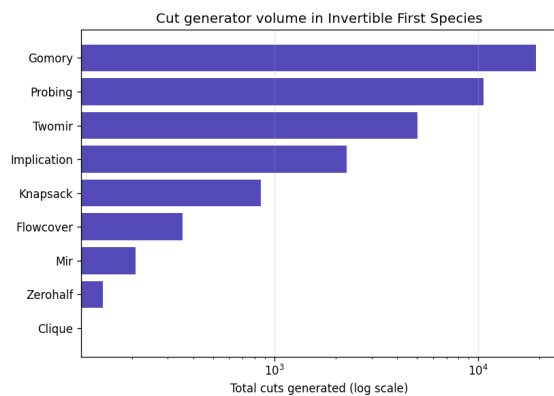
**Figure 7.** In second species, the time taken to solve each CF scales proportional to  $N^{5.84}$  following a power law.



**Figure 8.** A chart summarizing the number of times each cut was used during first species solves.



**Figure 9.** A chart summarizing the number of times each cut was used during second species solves.



**Figure 10.** A chart summarizing the number of times each cut was used during invertible first species solves.

firmus), while second species fares much worse — roughly  $N^{5.8}$  in our power-law fits versus  $N^{3.5}$  for first species and  $N^{2.6}$  for invertible first species. (We propose that the scaling for invertible species is much better due to our initial validation/rejection check, which immediately discards uncooperative counterpoints.) This is consistent with Tanaka’s hypothesis that the higher species may be too complex for integer programming to remain practical, and suggests that other declarative programming paradigms will likely outperform integer programming for third species and beyond. In particular, constraint logic programming provides an intuitive way for harmonic and melodic intervals to be completely determined by the pitches chosen for the cantus firmus and counterpoint at each time step. On the invertibility side, we found that requiring the cantus firmus to itself satisfy the counterpoint melodic rules is a good filter: the cantus firmi that ended with a 3–1 leap (and thus were not invertible) were immediately rejected by our solver.

Based on a power-law fit of our experimental data, we argue that the increase in complexity from first species to second species cannot be accounted for by the increased number of notes alone; instead, it is more likely that the dissonance-handling and strong-beat-weak-beat considerations really do add something “extra” to the complexity. For the above reasons, we conclude by not recommending the use of integer linear programming for the higher species of counterpoint, as the higher species introduce even more complex musical factors.

## 8. AI USAGE STATEMENT

We used a large language model to (i) generate 24 of the cantus firmi used as test inputs, (ii) assist in software engineering and documentation, and (iii) help polish this manuscript. All decisions about the integer-programming formulation, the rule set, and the experimental analysis were made by the authors. All generated text (for example, our `README.md`) was reviewed and revised by the authors prior to submission. The template used is that of the music computation conference ISMIR 2026.

## 9. REFERENCES

- [1] J. G. Hole, “Automatic species counterpoint:music generation at five levels using a guided local search algorithm,” 2021.
- [2] D. Herremans and K. Sørensen, “Composing first species counterpoint with a variable neighbourhood search algorithm,” *Journal of Mathematics and the Arts*, vol. 6, no. 4, pp. 169–189, 2012.
- [3] B. Schottstaedt, “Automatic species counterpoint,” Center for Computer Research in Music and Acoustics (CCRMA), Department of Music, Stanford University, Tech. Rep. STAN-M-19, May 1984. [Online]. Available: <https://ccrma.stanford.edu/files/papers/stanm19.pdf>

- [4] M. Komosinski and P. Szachewicz, “Automatic species counterpoint composition by means of the dominance relation,” *Journal of Mathematics and Music*, vol. 9, no. 1, pp. 75–94, 2015.
- [5] Y. Cong, “Weighted refinement types for counterpoint composition,” in *Proceedings of the 11th ACM SIG-PLAN International Workshop on Functional Art, Music, Modelling, and Design (FARM)*. Seattle, WA, USA: ACM, 2023, pp. 2–7.
- [6] T. Tanaka, “Formulating first species counterpoint with integer programming,” in *Proceedings of the 19th Sound and Music Computing Conference (SMC)*, Saint-Étienne, France, June 2022, pp. 601–607.
- [7] A. Zipris, “ILLIAC suite,” Illinois Distributed Museum, accessed: 2026-02-26. [Online]. Available: <https://distributedmuseum.illinois.edu/exhibit/illiac-suite/>
- [8] M. Farbood and B. Schoner, “Analysis and synthesis of Palestrina-style counterpoint using Markov chains,” in *Proceedings of the International Computer Music Conference (ICMC)*, Havana, Cuba, 2001. [Online]. Available: <https://alumni.media.mit.edu/~mary/palestrina.html>
- [9] F. Liang, M. Gotham, M. Johnson, and J. Shotton, “Automatic stylistic composition of Bach chorales with deep LSTM,” in *Proceedings of the 18th International Society for Music Information Retrieval Conference (ISMIR)*, Suzhou, China, 2017, pp. 449–456. [Online]. Available: [https://www.microsoft.com/en-us/research/wp-content/uploads/2017/11/156\\_Paper.pdf](https://www.microsoft.com/en-us/research/wp-content/uploads/2017/11/156_Paper.pdf)
- [10] J. J. Fux, *The Study of Counterpoint: From Johann Joseph Fux’s Gradus ad Parnassum*, revised ed., A. Mann, Ed. New York: W. W. Norton & Company, 1965, originally published in Latin in 1725.
- [11] J. Rodríguez Alvira, “Invertible counterpoint in Bach’s fugue BWV 885,” 2011. [Online]. Available: <https://www.teoria.com/en/articles/BWV885/index.php>